UNIVERSIDAD NACIONAL DE SAN AGUSTÍN DE AREQUIPA

FACULTAD DE INGENIERÍA, PRODUCCIÓN Y SERVICIOS

ESCUELA PROFESIONAL DE CIENCIA DE LA COMPUTACIÓN



CURSO:	COMPUTACIÓN PARALELA Y DISTRIBUIDA
TEMA:	PROGRAMACIÓN CON PTHREADS
DOCENTE:	MAMANI ALIAGA, ALVARO HENRY
ALUMNO:	QUINA DELGADO, MARCELO ADRIAN
FECHA:	27/10/2026

Arequipa - Perú

2026

1. Introducción y entorno de ejecución

En este informe resolví en C con Pthreads tres problemas, las estadísticas de un arreglo, la construcción de un histograma y el de productores y consumidores con buffer acotado. Para cada uno hice primero una versión secuencial y después la versión con threads, y comprobé que las dos den los mismos resultados.

1.1. Características del equipo

Componente	Detalle
Procesador	11th Gen Intel Core i7-11370H @ 3.30 GHz
Núcleos físicos	4
Hilos por núcleo	2
Hilos lógicos totales	8
Frecuencia máxima	4.8 GHz
Caché L2	5 MiB (4 instancias)
Caché L3	12 MiB
Memoria RAM	23 GiB
Sistema operativo	Ubuntu 24.04 (kernel 7.0.0-34-generic)
Compilador	GCC 13.3.0
Banderas de compilación	`-O2 -pthread`

1.2. Datos de las pruebas

Parte	Tamaño de los datos	Repeticiones por medición
I	N = 1 000 000, 10 000 000 y 100 000 000 enteros	5
II	N = 10 000 000 valores entre 0 y 99	3
III	100 000 elementos por productor	1

Los tiempos los medí con `clock_gettime` usando el reloj `CLOCK_MONOTONIC`, y cada tiempo que aparece en las tablas es el promedio de las repeticiones de arriba.

1.3. Compilación y ejecución

```
gcc -O2 -pthread parte1.c -o parte1_stats && ./parte1_stats
gcc -O2 -pthread parte2.c -o parte2_hist && ./parte2_hist
gcc -O2 -pthread parte3.c -o parte3_prodcons && ./parte3_prodcons
```

En la Parte I se pueden elegir los tamaños del arreglo, por ejemplo `./parte1_stats 1000000 10000000`, en la Parte II se puede elegir el N, por ejemplo `./parte2_hist 5000000` y en la Parte III se puede probar una configuración propia con `./parte3_prodcons P C B ITEMS`, donde P son los productores, C los consumidores, B el tamaño del buffer e ITEMS los elementos por productor.

2. Estadísticas del conjunto de datos

2.1. Implementar la solución secuencial.

Esta función recorre el arreglo una sola vez y va guardando la suma, el mínimo y el máximo.

```
static void estadisticas_secuencial(const int *arreglo, long n,
                                    long long *suma, int *minimo, int *maximo) {
    long long suma_local = 0;
    int minimo_local = arreglo[0];
    int maximo_local = arreglo[0];

    for (long i = 0; i < n; i++) {
        suma_local += arreglo[i];
        if (arreglo[i] < minimo_local) {
            minimo_local = arreglo[i];
        }
        if (arreglo[i] > maximo_local) {
            maximo_local = arreglo[i];
        }
    }
    *suma = suma_local;
    *minimo = minimo_local;
    *maximo = maximo_local;
}
```

2.2. Dividir el arreglo entre los threads, contemplando que N no sea divisible exactamente por la cantidad de threads.

A cada thread le toca un rango de posiciones desde inicio hasta fin sin incluir fin, y como N no siempre se divide exacto los primeros N mód p threads se llevan un elemento de más.

```
typedef struct {
    const int *arreglo;
    long inicio;
    long fin;
    long long suma_parcial;
    int minimo_parcial;
    int maximo_parcial;
} TareaThread;

/* n elementos entre T threads*/
static void repartir_trabajo(long n, int T, TareaThread *tareas, const int *arreglo) {
    long tamano_base = n / T;
    long sobrante = n % T;
    long posicion = 0;

    for (int i = 0; i < T; i++) {
        long tamano = tamano_base;
        if (i < sobrante) {
            tamano = tamano_base + 1;
        }
        tareas[i].arreglo = arreglo;
        tareas[i].inicio = posicion;
```

```
        tareas[i].fin = posicion + tamano;
        posicion = posicion + tamano;
    }
}
```

Thread	Inicio	Fin	Elementos
0	0	125001	125001
1	125001	250002	125001
2	250002	375003	125001
3	375003	500003	125000
4	500003	625003	125000
5	625003	750003	125000
6	750003	875003	125000
7	875003	1000003	125000

Así se reparten 1 000 003 elementos entre 8 threads, cada uno procesa las posiciones desde inicio hasta fin sin incluir fin.

2.3. Cada thread debe calcular resultados parciales usando variables locales.

Cada thread recorre solo su segmento y va acumulando en variables locales, y recién al final guarda los resultados en su estructura.

```
static void *calcular_segmento(void *argumento) {
    TareaThread *tarea = (TareaThread *)argumento;
    long long suma_local = 0;
    int minimo_local = tarea->arreglo[tarea->inicio];
    int maximo_local = tarea->arreglo[tarea->inicio];

    for (long i = tarea->inicio; i < tarea->fin; i++) {
        suma_local += tarea->arreglo[i];
        if (tarea->arreglo[i] < minimo_local) {
            minimo_local = tarea->arreglo[i];
        }
        if (tarea->arreglo[i] > maximo_local) {
            maximo_local = tarea->arreglo[i];
        }
    }
    tarea->suma_parcial = suma_local;
    tarea->minimo_parcial = minimo_local;
    tarea->maximo_parcial = maximo_local;
    return NULL;
}
```

2.4. Combinar los resultados parciales para obtener la suma, el mínimo y el máximo globales.

Primero se crean los threads, después se espera a que terminen con `pthread_join` y ahí el thread principal junta los resultados parciales.

```c
for (int i = 0; i < T; i++) {
    pthread_create(&threads[i], NULL, calcular_segmento, &tareas[i]);
}
for (int i = 0; i < T; i++) {
    pthread_join(threads[i], NULL);
}

long long suma_total = 0;
int minimo_global = tareas[0].minimo_parcial;
int maximo_global = tareas[0].maximo_parcial;

for (int i = 0; i < T; i++) {
    suma_total += tareas[i].suma_parcial;
    if (tareas[i].minimo_parcial < minimo_global) {
        minimo_global = tareas[i].minimo_parcial;
    }
    if (tareas[i].maximo_parcial > maximo_global) {
        maximo_global = tareas[i].maximo_parcial;
    }
}
*suma = suma_total;
*minimo = minimo_global;
*maximo = maximo_global;
```

2.5. Calcular el promedio a partir de la suma total y N.

```c
double promedio_sec = (double)suma_sec / n;
double promedio_par = (double)suma_par / n;
```

2.6. Comparar los resultados secuenciales y paralelos para varios tamaños de N.

Con 1, 2, 4 y 8 threads la versión paralela dio exactamente los mismos valores que la secuencial en los tres tamaños de N.

```c
int correcto = 0;
if (suma_par == suma_sec && minimo_par == minimo_sec &&
    maximo_par == maximo_sec && promedio_par == promedio_sec) {
    correcto = 1;
}
```

N	Suma	Mínimo	Máximo	Promedio	Tiempo (s)
1 000 000	370841016	-999996	999999	370.841016	0.001796
10 000 000	1291171948	-1000000	1000000	129.117195	0.008587
100 000 000	-886163492	-1000000	1000000	-8.861635	0.079201

Resultados de la versión secuencial para cada N.

2.7. Registrar los tiempos de ejecución y calcular speedup y eficiencia.

```c
static double tiempo_actual(void) {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return ts.tv_sec + ts.tv_nsec * 1e-9;
}

double tiempo_sec = 0;
for (int r = 0; r < REPETICIONES; r++) {
    double inicio = tiempo_actual();
    estadisticas_secuencial(arreglo, n, &suma_sec, &minimo_sec, &maximo_sec);
    tiempo_sec += tiempo_actual() - inicio;
}
tiempo_sec = tiempo_sec / REPETICIONES;

double tiempo_par = 0;

for (int r = 0; r < REPETICIONES; r++) {
    double inicio = tiempo_actual();
    estadisticas_paralela(arreglo, n, T, &suma_par, &minimo_par, &maximo_par);
    tiempo_par += tiempo_actual() - inicio;
}
tiempo_par = tiempo_par / REPETICIONES;

double speedup = tiempo_sec / tiempo_par;
double eficiencia = speedup / T;
```

N	Threads	T. sec. (s)	T. par. (s)	Speedup	Eficiencia	Correcto
1 000 000	1	0.001796	0.001912	0.939	0.939	Sí
1 000 000	2	0.001796	0.001044	1.721	0.861	Sí
1 000 000	4	0.001796	0.000855	2.101	0.525	Sí
1 000 000	8	0.001796	0.001019	1.764	0.220	Sí
10 000 000	1	0.008587	0.012102	0.710	0.710	Sí
10 000 000	2	0.008587	0.008553	1.004	0.502	Sí
10 000 000	4	0.008587	0.003549	2.420	0.605	Sí
10 000 000	8	0.008587	0.005128	1.674	0.209	Sí
100 000 000	1	0.079201	0.107558	0.736	0.736	Sí
100 000 000	2	0.079201	0.042295	1.873	0.936	Sí
100 000 000	4	0.079201	0.026747	2.961	0.740	Sí
100 000 000	8	0.079201	0.022558	3.511	0.439	Sí

Tiempos, speedup y eficiencia de la versión paralela, la columna correcto dice si la suma, el mínimo, el máximo y el promedio salieron iguales a los de la secuencial.

2.8. Preguntas de análisis

1. **¿Por qué conviene calcular resultados parciales en cada thread?**

   Porque así cada thread hace su parte con sus propias variables y no tiene que esperar a nadie, si todos usaran una sola variable se estarían estorbando entre ellos y casi sería como usar un solo thread, además nadie le borra el dato al otro y al final solo se juntan los resultados de cada uno que es rapidísimo comparado con recorrer todo el arreglo.

2. **¿Qué problema surgiría si todos los threads actualizaran una única variable de suma sin sincronización?**

   Pasaría que dos threads leen el mismo valor de la suma al mismo tiempo y cada uno le suma lo suyo y lo guarda, entonces uno de los dos aportes se pierde, eso es una condición de carrera y el resultado sale mal y además cambia en cada ejecución porque depende de quién llegue primero.

3. **¿Es necesario utilizar un mutex para combinar los resultados parciales? Justifique su estrategia.**

   No hizo falta porque cada thread guarda sus resultados solo en su propia estructura y la combinación la hace el thread principal después del `pthread_join`, o sea que cuando se combinan todos ya terminaron y nadie más toca esos datos, así que no hay nada que proteger y nos ahorramos el costo de usar un mutex.

4. **¿Cómo se distribuye el trabajo cuando N no es múltiplo del número de threads?**

   Se reparte casi parejo, a cada thread le toca N dividido entre el número de threads y lo que sobra se da de a uno a los primeros threads, por ejemplo con N = 1000003 y 8 threads sobran 3 así que los threads 0, 1 y 2 procesan 125001 elementos y los otros cinco 125000, de esa forma nadie tiene más de un elemento de diferencia.

5. **¿Por qué el speedup puede ser menor que el número de threads?**

   Porque crear los threads y juntar los resultados también toma tiempo y con arreglos chicos eso pesa mucho, además mi procesador tiene 4 núcleos físicos aunque muestre 8 hilos lógicos por eso con 8 threads no se duplica lo de 4, se ve en los resultados porque con 100 millones el speedup llegó a 3.5 y con un millón solo a 2.1.

3. Construcción del histograma

3.1. Implementar el histograma secuencial.

Para saber en qué categoría cae cada valor lo divido entre 10.

```
static void histograma_secuencial(const int *arreglo, long n, long *histograma) {
    memset(histograma, 0, CATEGORIAS * sizeof(long));
    for (long i = 0; i < n; i++) {
        int categoria = arreglo[i] / 10;
        histograma[categoria]++;
    }
}
```

3.2. Implementar una versión paralela en la que cada thread procese una parte del arreglo.

El arreglo se reparte igual que en la Parte I, y en esta primera versión todos los threads suman en el mismo histograma sin ninguna protección.

```
typedef struct {
    const int *arreglo;
    long inicio;
```

```
    long fin;
    long *histograma_compartido;
    long histograma_privado[CATEGORIAS];
    int estrategia;
} TareaThread;

/* reparto del arreglo*/
long tamano_base = n / T;
long sobrante = n % T;
long posicion = 0;
for (int i = 0; i < T; i++) {
    long tamano = tamano_base;
    if (i < sobrante) {
        tamano = tamano_base + 1;
    }
    tareas[i].arreglo = arreglo;
    tareas[i].inicio = posicion;
    tareas[i].fin = posicion + tamano;
    tareas[i].histograma_compartido = histograma;
    tareas[i].estrategia = estrategia;
    memset(tareas[i].histograma_privado, 0, sizeof(tareas[i].histograma_privado));
    posicion = posicion + tamano;
}

for (int i = 0; i < T; i++) {
    pthread_create(&threads[i], NULL, procesar_segmento, &tareas[i]);
}
for (int i = 0; i < T; i++) {
    pthread_join(threads[i], NULL);
}

static void *procesar_segmento(void *argumento) {
    TareaThread *tarea = (TareaThread *)argumento;

    for (long i = tarea->inicio; i < tarea->fin; i++) {
        int categoria = tarea->arreglo[i] / 10;

        switch (tarea->estrategia) {
        case SIN_SINCRONIZACION:
            tarea->histograma_compartido[categoria]++;
            break;
        }
    }
    return NULL;
}
```

3.3. Diseñar una estrategia para evitar que varios threads incrementen simultáneamente la misma posición del histograma.

Hice dos variantes con mutex, una con un mutex global para todo el histograma y otra con un mutex por cada categoría.

```
static pthread_mutex_t mutex_global = PTHREAD_MUTEX_INITIALIZER;
static pthread_mutex_t mutex_categoria[CATEGORIAS];

for (int c = 0; c < CATEGORIAS; c++) {
    pthread_mutex_init(&mutex_categoria[c], NULL);
```

```
}

case MUTEX_GLOBAL:
    pthread_mutex_lock(&mutex_global);
    tarea->histograma_compartido[categoria]++;
    pthread_mutex_unlock(&mutex_global);
    break;
case MUTEX_POR_CATEGORIA:
    pthread_mutex_lock(&mutex_categoria[categoria]);
    tarea->histograma_compartido[categoria]++;
    pthread_mutex_unlock(&mutex_categoria[categoria]);
    break;
```

3.4. Como alternativa, implementar un histograma privado por thread y combinar los histogramas al finalizar.

Acá cada thread cuenta en su propio histograma, que es el campo `histograma_privado` de su estructura, y cuando todos terminan el thread principal los suma.

```
case HISTOGRAMA_PRIVADO:
    /* cada thread cuenta en su propio histograma */
    tarea->histograma_privado[categoria]++;
    break;

/* histogramas privados se suman al final en el thread principal */
if (estrategia == HISTOGRAMA_PRIVADO) {
    for (int i = 0; i < T; i++) {
        for (int c = 0; c < CATEGORIAS; c++) {
            histograma[c] += tareas[i].histograma_privado[c];
        }
    }
}
```

3.5. Verificar que la suma de las diez categorías sea igual a N.

También compruebo que el histograma paralelo salga igual al secuencial.

```
static long sumar_categorias(const long *histograma) {
    long suma = 0;
    for (int c = 0; c < CATEGORIAS; c++) {
        suma += histograma[c];
    }
    return suma;
}

/* verificaciones */
long suma = sumar_categorias(hist_par);
int suma_correcta = 0;
if (suma == n) {
    suma_correcta = 1;
}
int igual_secuencial = 0;
if (memcmp(hist_sec, hist_par, sizeof(hist_sec)) == 0) {
    igual_secuencial = 1;
}
```

Distribución	0-9	10-19	20-29	30-39	40-49	50-59	60-69	70-79	80-89	90-99	T. (s)
Uniforme	999084	999296	999626	1000732	1000788	999693	1001211	1000831	999637	999102	0.010855
Sesgada	9100877	99778	99946	100104	99928	99419	99801	99843	99996	100308	0.023290
Constante	0	0	0	0	0	10000000	0	0	0	0	0.033146

Histograma secuencial de cada distribución con N = 10 000 000, en los tres casos las diez categorías suman N.

3.6. Comparar ambas estrategias paralelas, si se implementan, en términos de correctitud y tiempo.

Probé tres distribuciones de datos con 1, 2, 4 y 8 threads.

```
/* generacion de los datos segun la distribucion */
for (long i = 0; i < n; i++) {
    if (d == 0) {
        arreglo[i] = (int)(numero_aleatorio() % 100);
    } else if (d == 1) {
        if (numero_aleatorio() % 100 < 90) {
            arreglo[i] = (int)(numero_aleatorio() % 10);
        } else {
            arreglo[i] = (int)(numero_aleatorio() % 100);
        }
    } else {
        arreglo[i] = 55;
    }
}

double tiempo_par = 0;

for (int r = 0; r < REPETICIONES; r++) {
    double inicio = tiempo_actual();
    histograma_paralelo(arreglo, n, T, e, hist_par);
    tiempo_par += tiempo_actual() - inicio;
}
tiempo_par = tiempo_par / REPETICIONES;

double speedup = tiempo_sec / tiempo_par;
double eficiencia = speedup / T;
```

Estrategia	Threads	Tiempo (s)	Speedup	Eficiencia	Suma = N	Igual a sec.
Sin sincronización	1	0.028749	0.378	0.378	Sí	Sí
Sin sincronización	2	0.076587	0.142	0.071	No	No
Sin sincronización	4	0.075784	0.143	0.036	No	No
Sin sincronización	8	0.067473	0.161	0.020	No	No
Mutex global	1	0.189984	0.057	0.057	Sí	Sí
Mutex global	2	0.983480	0.011	0.006	Sí	Sí
Mutex global	4	0.989016	0.011	0.003	Sí	Sí
Mutex global	8	1.675498	0.006	0.001	Sí	Sí
Mutex por categoría	1	0.195963	0.055	0.055	Sí	Sí
Mutex por categoría	2	0.664787	0.016	0.008	Sí	Sí
Mutex por categoría	4	0.649318	0.017	0.004	Sí	Sí
Mutex por categoría	8	0.674745	0.016	0.002	Sí	Sí
Privado por thread	1	0.015562	0.698	0.698	Sí	Sí
Privado por thread	2	0.047639	0.228	0.114	Sí	Sí
Privado por thread	4	0.040335	0.269	0.067	Sí	Sí
Privado por thread	8	0.021228	0.511	0.064	Sí	Sí

Histograma con valores uniformes entre 0 y 99, el secuencial tardó 0.010855 s.

Estrategia	Threads	Tiempo (s)	Speedup	Eficiencia	Suma = N	Igual a sec.
Sin sincronización	1	0.039213	0.594	0.594	Sí	Sí
Sin sincronización	2	0.029294	0.795	0.398	No	No
Sin sincronización	4	0.021834	1.067	0.267	No	No
Sin sincronización	8	0.024858	0.937	0.117	No	No
Mutex global	1	0.200030	0.116	0.116	Sí	Sí
Mutex global	2	0.905143	0.026	0.013	Sí	Sí
Mutex global	4	0.936759	0.025	0.006	Sí	Sí
Mutex global	8	1.487670	0.016	0.002	Sí	Sí
Mutex por categoría	1	0.246563	0.094	0.094	Sí	Sí
Mutex por categoría	2	0.982801	0.024	0.012	Sí	Sí
Mutex por categoría	4	1.066157	0.022	0.005	Sí	Sí
Mutex por categoría	8	1.644007	0.014	0.002	Sí	Sí
Privado por thread	1	0.029932	0.778	0.778	Sí	Sí
Privado por thread	2	0.037994	0.613	0.306	Sí	Sí
Privado por thread	4	0.023764	0.980	0.245	Sí	Sí
Privado por thread	8	0.017568	1.326	0.166	Sí	Sí

Histograma con 90 % de los valores entre 0 y 9, el secuencial tardó 0.023290 s.

Estrategia	Threads	Tiempo (s)	Speedup	Eficiencia	Suma = N	Igual a sec.
Sin sincronización	1	0.044276	0.749	0.749	Sí	Sí
Sin sincronización	2	0.019568	1.694	0.847	No	No
Sin sincronización	4	0.011876	2.791	0.698	No	No
Sin sincronización	8	0.015165	2.186	0.273	No	No
Mutex global	1	0.199511	0.166	0.166	Sí	Sí
Mutex global	2	0.939571	0.035	0.018	Sí	Sí
Mutex global	4	0.974282	0.034	0.009	Sí	Sí
Mutex global	8	1.562459	0.021	0.003	Sí	Sí
Mutex por categoría	1	0.201237	0.165	0.165	Sí	Sí
Mutex por categoría	2	0.991256	0.033	0.017	Sí	Sí
Mutex por categoría	4	0.945842	0.035	0.009	Sí	Sí
Mutex por categoría	8	1.575193	0.021	0.003	Sí	Sí
Privado por thread	1	0.041581	0.797	0.797	Sí	Sí
Privado por thread	2	0.044441	0.746	0.373	Sí	Sí
Privado por thread	4	0.029576	1.121	0.280	Sí	Sí
Privado por thread	8	0.016057	2.064	0.258	Sí	Sí

Histograma con todos los valores iguales a 55, el secuencial tardó 0.033146 s.

Distribución	Threads	Suma obtenida	Incrementos perdidos
Uniforme	2	5331155	4668845
Uniforme	4	3348609	6651391
Uniforme	8	1425778	8574222
Sesgada	2	5623150	4376850
Sesgada	4	2803064	7196936
Sesgada	8	2268758	7731242
Constante	2	5395878	4604122
Constante	4	2587401	7412599
Constante	8	2270533	7729467

Incrementos que se perdieron por la condición de carrera en la versión sin sincronización con N = 10 000 000, con un solo thread no se pierde ninguno.

3.7. Preguntas de análisis

1. **¿En qué situación se produce una condición de carrera al actualizar el histograma?**

   Se produce cuando dos threads tienen valores de la misma categoría al mismo tiempo y los dos leen el contador le suman uno y lo guardan, entonces los dos escriben el mismo número y se perdió un incremento.

2. **¿Qué diferencias hay entre usar un mutex compartido y un histograma privado por thread?**

   Con el mutex compartido todos los threads usan el mismo histograma y tienen que turnarse en cada incremento así que pasan casi todo el tiempo esperando, con el

histograma privado cada thread cuenta en el suyo sin molestar a nadie y solo al final se suman los diez valores de cada uno.

3. **¿Qué estrategia esperarías que escale mejor y por qué?**

   Esperaba que escalara mejor el histograma privado porque los threads no se estorban y solo se juntan al final, con mutex mientras más threads hay más esperan y se nota en que el mutex global pasó de 0.19 s con 1 thread a 1.68 s con 8, eso sí en datos uniformes ni el privado le ganó al secuencial porque el trabajo por elemento es muy poco.

4. **¿Cómo influye la distribución de los valores de entrada en la contención?**

   Si los valores se reparten parejo entre las diez categorías hay menos choques por el mismo contador y si casi todos caen en la misma categoría todos los threads pelean por el mismo dato, se ve en el mutex por categoría que con 8 threads tardó 0.67 s con datos uniformes y 1.64 s con datos sesgados, en cambio el mutex global tardó parecido en todos porque de todas formas usan el mismo candado.

4. Productores y consumidores

4.1. Definir el buffer, los índices de inserción y extracción, y los contadores necesarios.

La estructura compartida tiene el buffer circular, los dos índices, el contador de elementos, los contadores para verificar, el mutex y las dos variables de condición.

```c
/* datos compartidos */
typedef struct {
    int *buffer;
    int capacidad;
    int indice_insercion;
    int indice_extraccion;
    int cantidad;               /* elementos actualmente en el buffer */
    long total_a_producir;
    long total_consumido;
    int max_ocupacion;
    int *veces_consumido;       /* cuantas veces salio el valor v */
    long long suma_consumida;
    pthread_mutex_t mutex;
    pthread_cond_t no_lleno;
    pthread_cond_t no_vacio;
} DatosCompartidos;

DatosCompartidos datos;
memset(&datos, 0, sizeof(datos));
datos.capacidad = B;
datos.buffer = malloc(B * sizeof(int));
datos.total_a_producir = (long)P * items;
datos.veces_consumido = calloc(datos.total_a_producir, sizeof(int));
pthread_mutex_init(&datos.mutex, NULL);
pthread_cond_init(&datos.no_lleno, NULL);
pthread_cond_init(&datos.no_vacio, NULL);
```

4.2. Crear al menos dos productores y dos consumidores.

```
for (int i = 0; i < C; i++) {
    args_consumidores[i].compartidos = &datos;
    args_consumidores[i].id = i;
    args_consumidores[i].cantidad_items = 0;
    args_consumidores[i].procesados = 0;
    args_consumidores[i].suma = 0;
    pthread_create(&threads_consumidores[i], NULL, consumidor, &args_consumidores[i]);
}
for (int i = 0; i < P; i++) {
    args_productores[i].compartidos = &datos;
    args_productores[i].id = i;
    args_productores[i].cantidad_items = items;
    args_productores[i].procesados = 0;
    args_productores[i].suma = 0;
    pthread_create(&threads_productores[i], NULL, productor, &args_productores[i]);
}
for (int i = 0; i < P; i++) {
    pthread_join(threads_productores[i], NULL);
}
for (int i = 0; i < C; i++) {
    pthread_join(threads_consumidores[i], NULL);
}
```

4.3. Proteger el acceso al buffer y sus índices para evitar modificaciones concurrentes inconsistentes.

```
pthread_mutex_lock(&datos->mutex);
/* espera si el buffer esta lleno */
datos->buffer[datos->indice_insercion] = valor;
datos->indice_insercion = (datos->indice_insercion + 1) % datos->capacidad;
datos->cantidad++;
if (datos->cantidad > datos->max_ocupacion) {
    datos->max_ocupacion = datos->cantidad;
}
/* aviso a los consumidores */
pthread_mutex_unlock(&datos->mutex);

pthread_mutex_lock(&datos->mutex);
/* espera si el buffer esta vacio */
int valor = datos->buffer[datos->indice_extraccion];
datos->indice_extraccion = (datos->indice_extraccion + 1) % datos->capacidad;
datos->cantidad--;
datos->total_consumido++;
datos->veces_consumido[valor]++;
datos->suma_consumida += valor;
/* avisos y termino */
pthread_mutex_unlock(&datos->mutex);
```

4.4. Utilizar mecanismos de sincronización apropiados para que los productores esperen cuando el buffer esté lleno y los consumidores cuando esté vacío.

```
while (datos->cantidad == datos->capacidad) {
    /* buffer lleno - el productor espera */
    pthread_cond_wait(&datos->no_lleno, &datos->mutex);
}

pthread_cond_signal(&datos->no_vacio);

while (datos->cantidad == 0 && datos->total_consumido < datos->total_a_producir) {
    /* buffer vacio - el consumidor espera */
    pthread_cond_wait(&datos->no_vacio, &datos->mutex);
}

if (datos->cantidad == 0) {
    /* ya se consumio todo - el consumidor termina */
    pthread_mutex_unlock(&datos->mutex);
    break;
}

pthread_cond_signal(&datos->no_lleno);

if (datos->total_consumido == datos->total_a_producir) {
    pthread_cond_broadcast(&datos->no_vacio);
}
```

4.5. Contabilizar los elementos producidos y consumidos y verificar que no se pierdan ni se dupliquen.

Cada valor que se produce es único, entonces cuento cuántas veces sale cada uno y comparo las cantidades y las sumas de lo producido y lo consumido.

```
/* conteo */
long producidos = 0;
long consumidos = 0;
long long suma_producida = 0;
for (int i = 0; i < P; i++) {
    producidos += args_productores[i].procesados;
    suma_producida += args_productores[i].suma;
}
for (int i = 0; i < C; i++) {
    consumidos += args_consumidores[i].procesados;
}

/* verificacion */
long no_exactos = 0;
for (long v = 0; v < datos.total_a_producir; v++) {
    if (datos.veces_consumido[v] != 1) {
        no_exactos++;
    }
}

int correcto = 1;
if (producidos != consumidos) {
    correcto = 0;
}
if (producidos != datos.total_a_producir) {
    correcto = 0;
```

```
}
if (no_exactos != 0) {
    correcto = 0;
}
if (suma_producida != datos.suma_consumida) {
    correcto = 0;
}
if (datos.cantidad != 0) {
    correcto = 0;
}
```

Prod.	Cons.	Buffer	Suma producida	Suma consumida	No exactos	Resto buffer
2	2	1	19999900000	19999900000	0	0
2	2	8	19999900000	19999900000	0	0
2	2	64	19999900000	19999900000	0	0
2	2	1024	19999900000	19999900000	0	0
4	4	8	79999800000	79999800000	0	0
4	4	64	79999800000	79999800000	0	0
1	4	16	4999950000	4999950000	0	0
4	1	16	79999800000	79999800000	0	0
8	8	32	319999600000	319999600000	0	0

En todas las pruebas la suma de lo producido es igual a la de lo consumido, ningún valor salió una cantidad de veces distinta de uno y el buffer terminó vacío.

4.6. Ejecutar pruebas con distintas cantidades de productores, consumidores y tamaños de buffer.

```
long items_por_productor = 100000;
int configuraciones[][3] = {
    {2, 2, 1}, {2, 2, 8}, {2, 2, 64},
    {4, 4, 8}, {4, 4, 64},
    {1, 4, 16}, {4, 1, 16},
    {8, 8, 32}, {2, 2, 1024}
};
int cantidad_pruebas = sizeof(configuraciones) / sizeof(configuraciones[0]);
int todas_correctas = 1;

for (int i = 0; i < cantidad_pruebas; i++) {
    int resultado = ejecutar_prueba(configuraciones[i][0], configuraciones[i][1],
                                    configuraciones[i][2], items_por_productor, 1);
    if (resultado == 0) {
        todas_correctas = 0;
    }
}
```

Prod.	Cons.	Buffer	Esperados	Producidos	Consumidos	Máx. ocup.	Tiempo (s)	Correcto
2	2	1	200000	200000	200000	1	1.2402	Sí
2	2	8	200000	200000	200000	8	0.1954	Sí
2	2	64	200000	200000	200000	64	0.0682	Sí
2	2	1024	200000	200000	200000	1024	0.0554	Sí
4	4	8	400000	400000	400000	8	0.4931	Sí
4	4	64	400000	400000	400000	64	0.2119	Sí
1	4	16	100000	100000	100000	16	0.0775	Sí
4	1	16	400000	400000	400000	16	0.2884	Sí
8	8	32	800000	800000	800000	32	0.6149	Sí

Resultados de las pruebas con distintos productores, consumidores y tamaños de buffer, con 100 000 elementos por productor.

Prod.	Cons.	Buffer	Elementos por consumidor
2	2	1	100002, 99998
2	2	8	110159, 89841
2	2	64	110173, 89827
2	2	1024	96090, 103910
4	4	8	101111, 100169, 99999, 98721
4	4	64	95703, 99917, 100370, 104010
1	4	16	33565, 23919, 18824, 23692
4	1	16	400000
8	8	32	100922, 100464, 99367, 100111, 100210, 100228, 98127, 100571

Cuántos elementos sacó cada consumidor en cada prueba.

4.7. Preguntas de análisis

1. **¿Por qué no basta con proteger únicamente el contador de elementos?**

   Porque el buffer y los índices también los modifican varios threads a la vez y si no se protegen dos productores pueden escribir en la misma posición y uno borra el dato del otro, o dos consumidores sacan el mismo elemento, aunque el contador esté bien igual se pierden o se duplican datos.

2. **¿Qué puede ocurrir si un consumidor intenta extraer un elemento de un buffer vacío?**

   Sacaría un dato que no existe, o sea basura o un valor viejo que ya se había consumido, y además los índices y el contador quedarían mal, por eso el consumidor espera con la variable de condición hasta que un productor meta algo y recién ahí saca el elemento.

3. **¿Qué puede ocurrir si un productor inserta en un buffer lleno?**

   Se sobrescribiría un dato que todavía nadie consumió y ese elemento se perdería sin que nadie se dé cuenta, además el contador pasaría de la capacidad del buffer, por eso el productor espera hasta que un consumidor libere un espacio.

4. **¿Qué función cumplen el mutex y las variables de condición o semáforos elegidos?**

   El mutex hace que solo un thread a la vez toque el buffer y los índices y los contadores, las variables de condición sirven para dormir a un thread cuando no puede avanzar, el productor si el buffer está lleno y el consumidor si está vacío, y se despiertan cuando algo cambia así no gastan procesador preguntando todo el rato.

5. **¿Cómo comprobarías que todos los elementos producidos fueron consumidos exactamente una vez?**

   Cada valor producido es único así que uso un arreglo que cuenta cuántas veces salió cada valor y al final reviso que todos valgan exactamente uno, además comparo que producidos y consumidos sean iguales, que la suma de los valores producidos sea igual a la de los consumidos y que el buffer termine vacío.

5. Resultados y análisis

N	Mejor speedup	Threads	Eficiencia
1 000 000	2.101	4	0.525
10 000 000	2.420	4	0.605
100 000 000	3.511	8	0.439

Mejor speedup de la Parte I para cada N.

Distribución	Secuencial	Sin sincr.	Mutex global	Mutex por cat.	Privado
Uniforme	0.010855	0.067473	1.675498	0.674745	0.021228
Sesgada	0.023290	0.024858	1.487670	1.644007	0.017568
Constante	0.033146	0.015165	1.562459	1.575193	0.016057

Tiempos en segundos de la Parte II con 8 threads para cada estrategia y distribución. La versión sin sincronización da resultados incorrectos.

Parte I. El speedup mejora mientras más grande es el arreglo, el mejor fue 2.1 con un millón de datos, 2.4 con diez millones y 3.5 con cien millones, y la eficiencia baja al subir los threads porque mi equipo tiene 4 núcleos físicos, con un solo thread la versión paralela salió más lenta que la secuencial por el costo de crear el thread, y en todos los casos los resultados fueron iguales a los secuenciales.

Parte II. La versión sin sincronización dio resultados malos desde 2 threads y no sirve aunque sea rápida, los dos mutex dieron resultados correctos pero lentísimos, con 8 threads y datos uniformes el mutex global tardó unas 150 veces más que el secuencial. El histograma privado fue correcto y el más rápido de los paralelos correctos, y con datos sesgados y constantes llegó a ganarle al secuencial con speedup de 1.3 y 2.1 con 8 threads.

Parte III. Todas las pruebas salieron correctas sin elementos perdidos ni repetidos, el tamaño del buffer cambió mucho el tiempo, con 2 productores y 2 consumidores el buffer de 1 posición tardó 1.24 s y el de 1024 solo 0.055 s. Los consumidores no se repartieron el trabajo parejo.

6. Problemas encontrados y soluciones

El primer problema fue la condición de carrera en la versión sin sincronización del histograma, con 2 o más threads el resultado salía mal y cambiaba en cada ejecución, y se resolvió con mutex y con histogramas privados que sí dieron la suma exacta de N, pero los mutex dejaron todo muchísimo más lento y por eso el histograma privado terminó siendo la mejor opción.

En la Parte I la versión paralela con un thread salió más lenta que la secuencial, eso no es un error sino el costo de crear el thread, y para que los tiempos no dependieran de una sola corrida se promediaron 5 repeticiones en la Parte I y 3 en la Parte II.

En productores y consumidores había que cuidar cómo terminan los consumidores, porque cuando ya no quedan datos los que están esperando se quedarían dormidos para siempre, y se resolvió con un `pthread_cond_broadcast` cuando se consume el último elemento.

7. Conclusiones

Correctitud. Todas las versiones paralelas dieron el mismo resultado que la secuencial, menos la del histograma sin sincronización que perdió millones de incrementos por la condición de carrera, en productores y consumidores no se perdió ni se repitió ningún elemento en las 9 pruebas.

Sincronización. Sincronizar es obligatorio cuando varios threads tocan el mismo dato pero cuesta, el mutex global y el mutex por categoría fueron correctos pero lentísimos, lo mejor fue evitar compartir con resultados parciales y con histogramas privados que solo se juntan al final y cuando un thread tiene que esperar se usan variables de condición en vez de preguntar todo el tiempo.

Rendimiento. Más threads no siempre es más rápido el speedup depende del tamaño de los datos de los núcleos del equipo y de cuánto hay que sincronizar, el mejor caso fue 3.5 con 8 threads y cien millones de datos, y en el histograma el privado fue el único de los correctos que le ganó al secuencial en algunos casos. Importó más cómo se reparte el trabajo y cómo se evitan las esperas que solo agregar más threads.

Codigo fuente:
`https://github.com/MarceloMAQD/LAB_Pthreads_paralela.git`